

* Pandas *

H

Introduction to Pandas:-

- Pandas is python library designed to work with data sets - It offers robust tools for analyzing, cleaning, exploring, and manipulating data.

- The name "pandas" is derived from "panel Data" and "python Data Analysis", and it was created by Wes McKinney in 2008.

Why Use Pandas?

- Enables analysis of large datasets to draw conclusions based on statistical reasoning.

- Capable of cleaning messy datasets, transforming them into readable and relevant formats - critical in data science workflows.

Core Capabilities:-

- Pandas helps answer essential data-related questions:-
like,

* Correlation between columns.

* Statistical metrics like average, maximum, and minimum values.

* Data Cleaning, such as removing rows with missing or irrelevant values (e.g. empty or NULL entries)

Installation:-

- Ensure Python and pip are installed, then install pandas using:

```
"pip install pandas"
```

Importing Pandas:-

① Basic import:-

```
"import pandas"
```

After this, you can use functions like pandas.
DataFrame().

② Common Convention: import with alias **pd** for brevity.

```
import pandas as pd
```

```
myvar = pd.DataFrame(mydataset)
```

* Using **pd** streamlines code referencing.

Creating a DataFrame:-

```
mydataset = {
```

```
    "cars": ["BMW", "volvo", "Ford"],
```

```
    'passing': [3, 7, 2]
```

```
}
```

```
myvar = pd.DataFrame(mydataset)
```

```
print(myvar)
```

Output →

	cars	passings
0	BMW	3
1	Volvo	7
2	Ford	2

- This produces a simple tabular structure with labeled columns (cars, passings).

Checking Pandas Version:-

- To verify which version of Pandas you're working with,

`print(pd.__version__)`

- This can be useful for ensuring compatibility or leveraging version-specific features.

Pandas Series:-

① What is a Series?

- A Pandas Series is one-dimensional array - similar to a column in a table - that can hold data of any type.

② Labels (Index):-

- By default, Series are labeled with integer indices starting at 0, 1, 2, etc.

- You can access values using these indices, e.g., `myvar[0]`

② Custom Labels with index:-

- you can define custom labels using the index parameter upon creation,

```
myvar = pd.Series([1, 2, 3], index = ["x", "y", "z"])
```

- Then, access elements using labels:-

```
myvar["y"]
```

③ Creating a Series from a Dictionary:-

- A dictionary can be used to create a Series, where keys become labels and values become data.

```
calories = {"day1": 420, "day2": 380, "day3": 390}
myvar = pd.Series(calories)
```

- you can also specify keys via the index argument,

```
myvar = pd.Series(calories, index = ["day1", "day2"])
```

④ Relationship with Dataframe:-

- A Series represents one column of data.

- A Dataframe is a two-dimensional structure (table) composed of multiple Series.

Summary Table:-

Defination → One-dimensional labelled array (akin to a table column)

Default indexing → Integer-based (0, 1, 2, ...)

Custom Labels (index) → Use descriptive labels like "x", "y", "z"

Dictionary to Series → Keys become labels, values become data.

Selective Indexing → Use `index = [...]` to filter items from a dictionary.

Relation to DataFrame → Series = one column,
DataFrame = multiple series (like a full table)

Pandas DataFrame:-

What is a DataFrame?

- A Pandas DataFrame is a two-dimensional data structure - similar to a table - with labeled rows and columns, much like a spreadsheet or SQL table.

Creating a DataFrame:-

- You can create a DataFrame using a dictionary of lists.

```
import pandas as pd
```

```
data = {
```

```
    "colonies": [420, 380, 390],
```

```
    "duration": [50, 40, 45]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

- This yields a table with two columns - colonies and duration - and automatically numbered row indices starting at 0.

⑧ Locating Rows Using loc :-

- Single row :-

Use `df.loc[0]` to return row with index 0 as a Series.

- Multiple rows :-

Use a list like `df.loc[[0, 1]]` to return a DataFrame with the specified rows.

⑧ Custom Row Indexes :-

- You can assign custom labels to rows using the `index` parameter.

```
df = pd.DataFrame(
```

```
    data,
```

```
    index = ["day1", "day2", "day3"]
```

- This results in row labels like day1, day2, day3.

② Accessing Rows by Label:-

- Once custom indexes are assigned, you can select rows by label.

```
print(df.loc["day2"])
```

- This returns the row corresponding to day2 as a series.

② Summary Table:-

Defination → A 2D labeled data structure (tabular form)

Creation → Via (a dictionary of columns → `pd.DataFrame(data)`)

Default indexing → Integer indices (0, 1, 2, ...)

Row Access → `df.loc[0]` → Series,
`df.loc[[0, 1]]` → DataFrame.

Custom Row labels → `index = ["day1", "day2", ...]`

Label-based Access → `df.loc["day2"]` returns specific labeled row.

#

Pandas Read CSV:-

① Introduction to CSV:-

- CSV (comma-separated values) is a plain-text format commonly used for storing large datasets, and it's fully supported by pandas.

② Reading a CSV into a DataFrame:-

- Load a CSV file into a pandas DataFrame with,

```
import pandas as pd  
df = pd.read_csv('data.csv')
```

- To view the entire DataFrame in the console use,

```
print(df.to_string())
```

This prints all rows without truncation.

③ Default Display Behavior:-

- By default, printing a large DataFrame like `print(df)` only shows the first 5 rows & last 5 rows.

- You can inspect or modify how many rows pandas displays using display options.

① Managing Display setting:-

- check the current maximum rows display limit with:

```
* print(pd.options.display.max_rows)
```

- Increase the limit to show more rows,

```
* pd.options.display.max_rows = 9999
```

- This setting ensures that `print(df)` displays the entire DataFrame without truncation.

Pandas Read JSON :-

① Introduction to JSON:-

- JSON (JavaScript Object Notation) is a lightweight, plain-text format commonly used for storing and transporting data.

- Large data sets are frequently stored or received in JSON format and are easily processed using pandas.

① Reading JSON Files:-

- To load a JSON file into a Pandas DataFrame, use,

```
import pandas as pd
```

```
df = pd.read_json('data.json')
```

```
print(df.to_string())
```

- The use of `to_string()` prints the complete DataFrame without truncation - ideal for full data visibility.

② Using Dictionaries as JSON!:-

- Since JSON objects share the same structure as python dictionaries, you can create a DataFrame directly from dict,

```
import pandas as pd
data = {
    "Duration": {"0": 60, "1": 60, "2": 60, "3": 45, "4": 45},
    "pulse": {"0": 110, "1": 117, "2": 103, "3": 109, "4": 117}
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

- Here, dictionary keys map to column names & nested dictionaries use their keys as row labels.

Pandas Analyzing Dataframes:-

② Viewing a Snapshot of the DataFrame:

`df.head(n)` returns the first n rows (including headers). If no number is specified, it defaults to 5

- `df.tail(n)` returns the last `n` rows (also with headers), Defaults to the last 5 rows if no number is provided.

② Inspecting Dataframe Structure with `.info()`:-

- `df.info()` prints essential metadata.

- * The range index (total rows)

- * Total columns and their names

- * Non-null counts per column.

- * Data types of each column.

- * Memory usage of the Dataframe.

③ Spotting Missing Values:-

- The non-null count shown by `.info()` helps identify missing or null entries.

- for example if the "calories" column shows 169 non-null entries out of 169, it implies 5 rows are missing value.

- Detecting these gaps early is key for preparing clean, reliable data for analysis.

Pandas Cleaning Data:-

① What is Data cleaning?

- Data cleaning involves fixing "bad" data in your dataset, such as:

- * Empty cells
- * Wrong data formats.
- * Incorrect data values.
- * Duplicate entries.

② Cleaning Empty Cells:-

- Remove rows with empty values using.

- * `df.dropna()`
- * `df.dropna(inplace=True)`

- The default without `inplace=True`, returns a new cleaned DataFrame.

- To modify the original DataFrame directly add `inplace=True` argument.

③ Why Handle Empty cells?

- Empty cells (NaN & NULL) values can skew your data analysis and lead to inaccurate results.

① Replacing Empty cells with Default Values:-

- Instead of dropping rows, you can fill missing entries using `fillna()`,

```
* df.fillna(130, inplace=True)
```

- This replaces all missing values in the DataFrame with 130.

- To target a specific column, use a dictionary.

```
* df.fillna({"calories": 130}, inplace=True)
```

② Filling Empty Cells with Statistical Values:-

- A more intelligent approach is to fill missing values based on columns statistics.

*** Mean (average):-**

```
x = df["calories"].mean()
```

```
df.fillna({"calories": x}, inplace=True)
```

*** Median (middle value):-**

```
x = df["calories"].median()
```

```
df.fillna({"calories": x}, inplace=True)
```

*** Mode (most frequent value):-**

```
x = df["calories"].mode()[0]
```

```
df.fillna({"calories": x}, inplace=True)
```

Pandas: Cleaning 'wrong' format:-

① What is "wrong format"?

- cells may contain valid-looking data but in inconsistent or incorrect formats (e.g. mismatched date strings). This makes analysis difficult or unreliable.

② Two ways to fix it:-

- Remove the problematic rows, or
- Convert all entries in the columns to a standard format for consistency.

③ Converting to correct format with `to_datetime()`:-

- Example Scenario. The Date column has formatting issues. row 22 is empty (NaN), and row 26 is incorrectly formatted (20201226 without separators)

- fix with pandas

```
* df['Date'] = pd.to_datetime(df['Date'], format='mixed')  
* print(df.to_string())
```

- The valid date strings are converted successfully.

- The incorrectly formatted entry 20201226 becomes a proper datetime object.

- The empty cell is converted ^{to} NaT ("Not a Time"), representing a missing date.

① Removing Rows with Invalid Dates:-

- After conversion, you can drop rows where Date is NaT

* `df.dropna(subset=['Date'], inplace=True)`

- This removes rows with null or invalid date entries.

Pandas Cleaning Wrong Data:-

① What Counts as "Wrong Data"?

- "Wrong data" refers to values that are not technically missing or in the wrong format, but logically incorrect - like a workout duration of 450 minutes when most durations are between 30 of 60

① Method to Address Wrong Data:-

Manual Replacement,

- Useful for small datasets or obvious typos.

- Example - fixing row 7's duration from 450 to a more realistic 45,

```
* df.loc[7, 'Duration'] = 45
```

Rule-Based Correction,

- Create logic to handle outliers across the dataset.

- Example: If any duration exceeds 120 minutes, cap it at 120.

```
for x in df.index:
```

```
    if df.loc[x, "Duration"] > 120:
```

```
        df.loc[x, "Duration"] = 120
```

Removing Erroneous Rows,

- Sometimes easier to drop problematic rows than correct them, especially if they're few & not critical.

```
for x in df.index:
```

```
    if df.loc[x, "Duration"] > 120:
```

```
        df.drop(x, inplace=True)
```

Pandas Removing Duplicates:

① Identifying Duplicate Rows with `df.duplicated()` :-

- The `df.duplicated()` method returns a Boolean Series, where each True indicates a duplicate row (beyond the first occurrence), and False indicates a unique one.

- Useful for flagging duplicate entries;

* `df.duplicated()`

- This method includes parameters like `subset` (to check specific columns) and `keep` (to define which duplicates to keep.)

② Removing Duplicate Rows with `drop_duplicates()`

- Use `df.drop_duplicates()` to remove duplicate rows, retaining just the first occurrence by default.

e.g.

* `df.drop_duplicates(inplace=True)`

- This modifies the original DataFrame.

Optional parameters include,

- `subset` - Specify particular column(s) to consider when identifying duplicates.

- keep :- choose 'first', 'last', or 'false' to control which duplicates to keep.

- inplace :- if True, performs the action in place, otherwise, returns a new Dataframe

Pandas Correlations:-

① Purpose of corr():-

- Pandas provides the corr() method to measure the relationship between numeric columns in a Dataframe. It generates a correlation matrix that quantifies how closely two variable move together.

② How to use it:-

* df.corr()

- This call returns a table where each cell indicates the correlation coefficient between two columns. Non-numeric columns are automatically excluded.

③ Interpreting Correlation Values:-

- The values range between -1 & 1

- 1 means a perfect positive correlation. both variable increases together

- "-1" indicates a perfect negative correlation, as one increases, the other decreases.

- "0" implies no linear correlation

④ Example output :-

- Suppose your DataFrame includes these columns: Duration, pulse, maxpulse & calories. The output might look like

	Duration	pulse	max pulse	calories
Duration	1.00	-0.155	0.00	0.92
pulse	-0.15	1.00	0.78	0.02
maxpulse	0.00	0.78	1.00	0.20
calories	0.92	0.02	0.20	1.00

- perfect correlation :- Duration vs Duration is always 1

- Strong positive correlation :- Duration vs calories is ~ 0.92 - suggesting that longer exercise sessions tend to burn more calories.

- Weak correlation :- Duration vs maxpulse is near zero (~ 0.009), implying little predictive association between them.

② Advanced Usage Choosing Methods & settings:-

Syntax:-

* `Dataframe.corr(method = 'pearson', min_periods = None)`

- Method:- choose between statistical technique:- 'pearson' (default), 'kendall', or 'spearman'

- min_periods:- Minimum number of valid observations required for the correlation calculations.

② Context Matters - Correlation $\neq$ Causation,

- A high correlation doesn't necessarily mean one variable causes another - they only move together statistically.

- It's important to critically evaluate whether relationships are causal when making conclusions.

② Summary Table:-

Method $\rightarrow$ `df.corr()` generates correlation matrix

DataType → only numeric columns are included.

Value Range → from -1 (Negative) to 1 (positive); 0 = no linear relationship

Common Use → Identify strong, weak, or inverse relationship between variables.

Advanced Parameters → method (pearson, kendall, spearman), min_periods.

Interpretation → Correlation does not imply causation.
Caveat

Pivoting & Reshaping :-

● **pivot Table** :-

- Reshape data like Excel pivot tables,

```
import pandas as pd
```

```
data = {
```

```
    'city': ['Paris', 'Paris', 'London', 'London']
```

```
    'Year': [2020, 2021, 2020, 2021],
```

```
    'Sales': [200, 250, 300, 400]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
pivot = df.pivot_table(values='Sales', index='city', columns='Year',  
                        aggfunc='sum')
```

```
print(pivot)
```

Output:

	Year	2020	2021
city			
London		300	400
Paris		200	250

③ Melt (Unpivot):

- Convert wide format to long format

```
df = pd.DataFrame({  
    'city': ['Paris', 'London'],  
    '2020': [200, 300],  
    '2021': [250, 400]  
})
```

```
melted = pd.melt(df, id_vars=['city'], var_name='year',  
                 value_name='sales')
```

```
print(melted)
```

Output:

	city	Year	sales
0	Paris	2020	200
1	London	2020	300
2	Paris	2021	250
3	London	2021	400

Indexing:-

① Set Index:-

- Use a column as index

```
df = pd.DataFrame({
    'ID': [1, 2, 3],
    'Name': ['Tom', 'Jerry', 'Spike']
})
```

```
df2 = df.set_index('ID')
```

output:

	ID	Name
	1	Tom
	2	Jerry
	3	Spike

② Reset Index:-

- Bring index back as a column.

```
print(df2.reset_index())
```

output:

	ID	Name
0	1	Tom
1	2	Jerry
2	3	Spike

Merging & Joining:-

① Merge:-

- Combine two Dataframes on a column.

```
df1 = pd.DataFrame({'ID': [1, 2], 'Name': ['Tom', 'Jerry']})
```

```
df2 = pd.DataFrame({'ID': [1, 2], 'Age': [25, 30]})
```

```
merged = pd.merge(df1, df2, on='ID')
```

```
print(merged)
```

Output:

	ID	Name	Age
0	1	Tom	25
1	2	Jerry	30

② Concat:-

- Stack Dataframe together.

```
df1 = pd.DataFrame({'A': [1, 2]})
```

```
df2 = pd.DataFrame({'A': [3, 4]})
```

```
print(pd.concat([df1, df2]))
```

Output:

	A
0	1
1	2
0	3
1	4

Grouping & Aggregation:

① GroupBy:-

- Summarize data by groups.

```
df = pd.DataFrame({
    'city': ['paris', 'paris', 'London', 'London'],
    'sales': [200, 250, 300, 400]
})
```

```
print(df.groupby('city')['sales'].sum())
```

Output:

city

London 700

paris 450

Name: sales, dtype: int64

Filtering & Boolean Indexing:

① Boolean Filter:-

```
df = pd.DataFrame({'Name': ['Tom', 'Jerry', 'Spike'], 'Age': [25, 12, 30]})
```

```
print(df[df['Age'] > 20])
```

① Output:

	Name	Age
0	Tom	25
2	Spike	30

isin:-

- Check membership.

```
print(df[df['Name'].isin(['Tom', 'Spike'])])
```

Output:

	Name	Age
0	Tom	25
2	Spike	30

#

Handling Missing Data (Advanced)

① forward fill (ffill):-

- fill missing values with previous one.

```
df = pd.DataFrame({'A': [1, None, 3, None, 5]})  
print(df.fillna(method='ffill'))
```

Output:

	A
0	1.0
1	1.0
2	3.0
3	3.0
4	5.0

① Interpolate:-

- fill missing values mathematically.

```
df = pd.DataFrame({'A': [1, None, 3, None, 5]})
print(df.interpolate())
```

Output:

	A
0	1.0
1	2.0
2	3.0
3	4.0
4	5.0

Apply & custom function

① Apply function:-

Run custom function on columns.

```
df = pd.DataFrame({'A': [1, 2, 3]})
print(df['A'].apply(lambda x: x**2))
```

Output:

0	1
1	4
2	9

Name: A, dtype: int64

#

Statistical functions:-

① Describe:-

- Get summary stats.

```
df = pd.DataFrame({'A': [10, 20, 30, 40, 50]})
print(df.describe())
```

Output:

	A
count	5.00
mean	30.00
std	15.81
min	10.00
25%	20.00
50%	30.00
75%	40.00
max	50.00

#

Time Series:-

① to_datetime

- Convert string to dates.

```
df = pd.DataFrame({'Date': ['2020-01-01', '2020-01-02', '2020-01-03'],
                  'Sales': [200, 300, 400]})
df['Date'] = pd.to_datetime(df['Date'])
print(df.dtypes)
```

Output:

```
Date      datetime64[ns]
Sales      int64
dtype: object
```

⊗ Resample

Aggregate by time period.

```
dates = pd.date_range('2020-01-01', periods=4, freq='D')
df = pd.DataFrame({'Date': dates, 'Sales': [200, 300, 400, 500]})
df = df.set_index('Date')
print(df.resample('2D').sum())
```

Output:

	Sales
Date	
2020-01-01	500
2020-01-03	900

Data Types & Conversion:-

⊗ astype

- change column data type.

```
df = pd.DataFrame({'A': ['1', '2', '3']})
df['A'] = df['A'].astype(int)
print(df.dtypes)
```

Output:

A int32
dtype: object

Array of time period.

df = pd.DataFrame({'date': date, 'total': total})
df = df.set_index('date')
df['total'].resample('D').sum()

Output:

2012
2013
2014

Date type conversion

Output:

Array of time period.

df = pd.DataFrame({'date': date, 'total': total})
df = df.set_index('date')
df['total'].resample('D').sum()